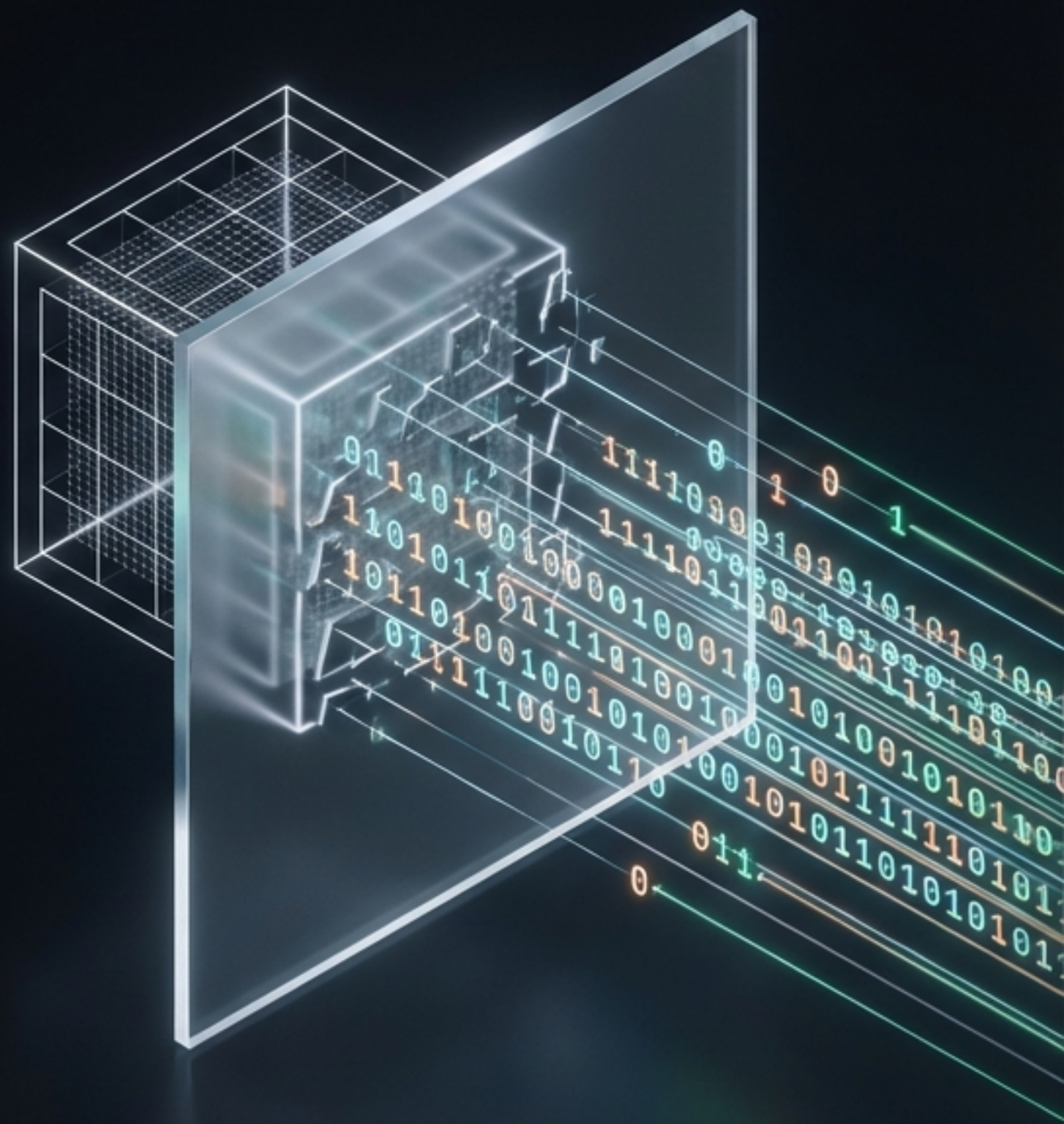


Архітектура **Серіалізації**: Від Байтів до Контрактів

Системний погляд на збереження, передачу
та еволюцію даних.



InterDisplay: Збереження в Базі Даних (Persistence Boundary)

SQL Mapping

- Колонки: Формальна специфікація
- Транзакційна дисципліна
- Прозора аналітика

Serialized

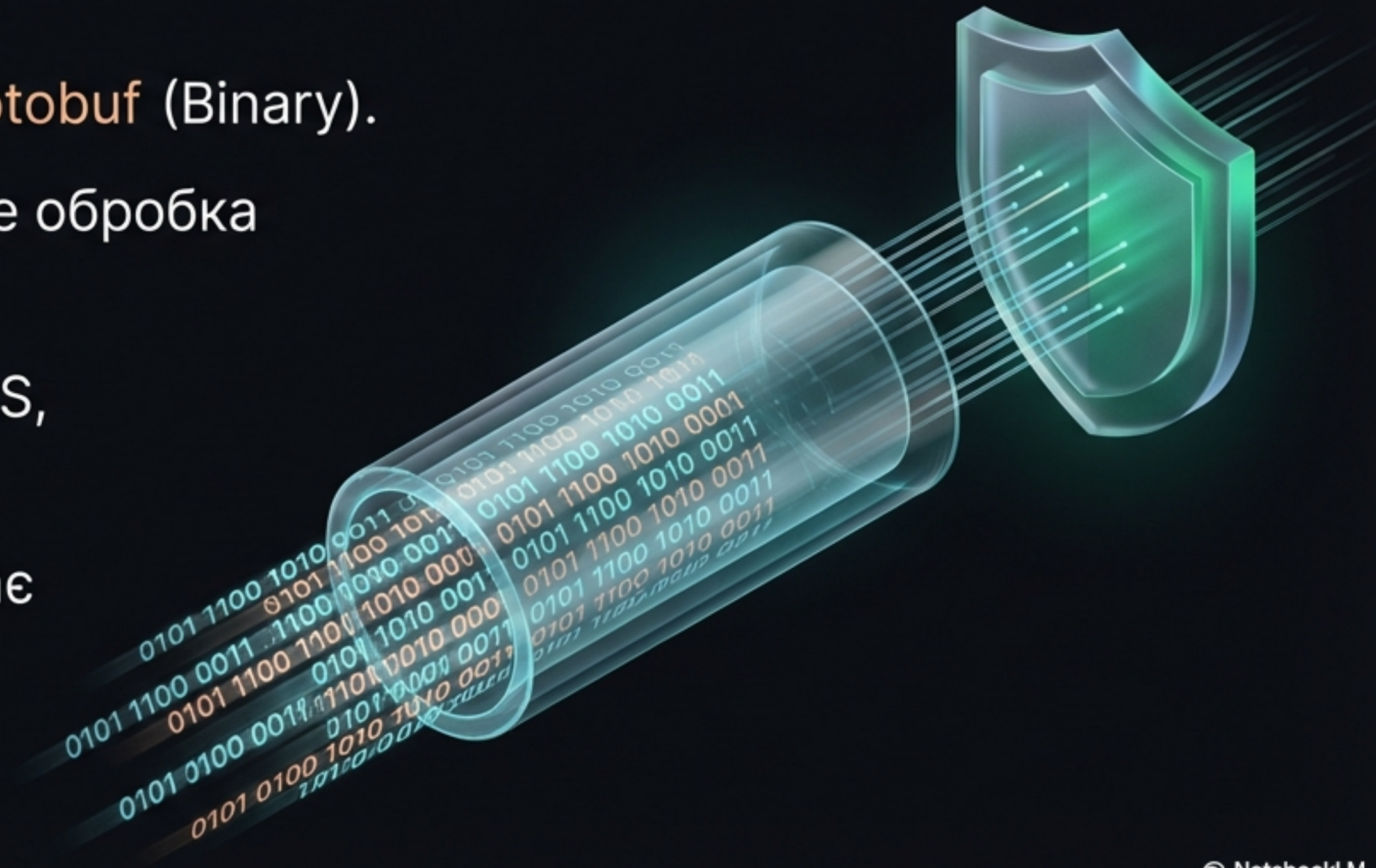
- Агрегат (JSONB/Blob)
- Гнучкість («схема в голові»)
- Технічний борг

-- Columnar vs Serialized
id: INTEGER, name: TEXT
vs
data: JSONB

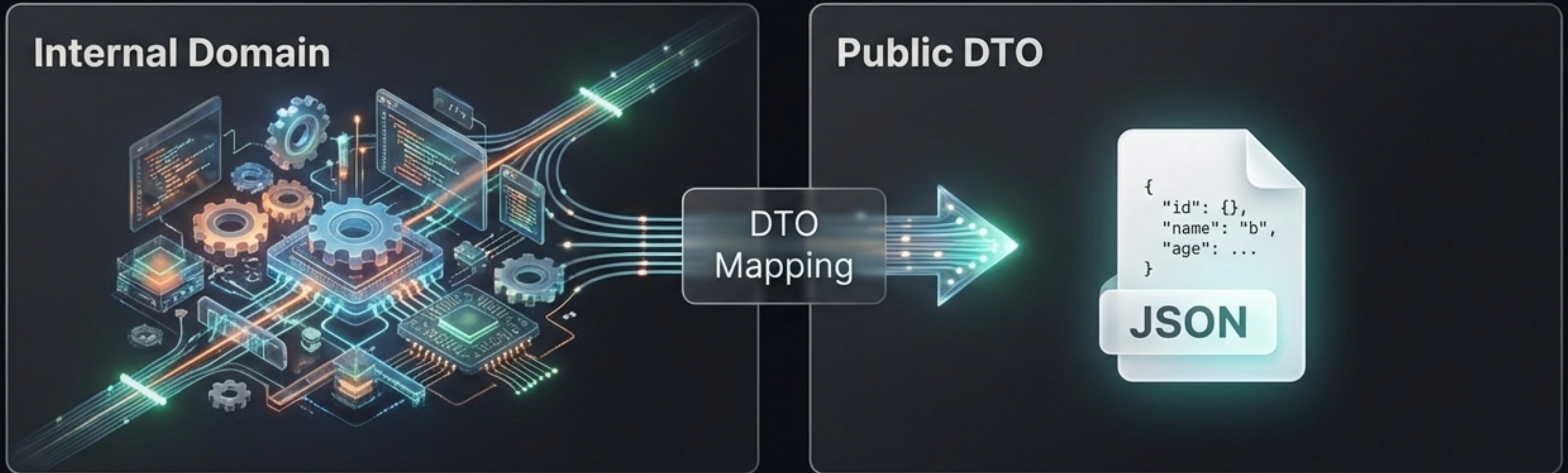
Проблема: `struct Person` не можна зберегти «як є». Потрібен адаптер.

Передача по Мережі (The Hostile Medium)

- **Реальність:** Транспорт (HTTP, gRPC) = Байтові потоки.
- **Формати:** JSON (Text) vs Protobuf (Binary).
- **Безпека:** Десеріалізація — це обробка недовіреного входу.
 - Ризики: Partial payload, DoS, логічні уразливості.
- **Контракт:** Формат байтів стає публічним API.

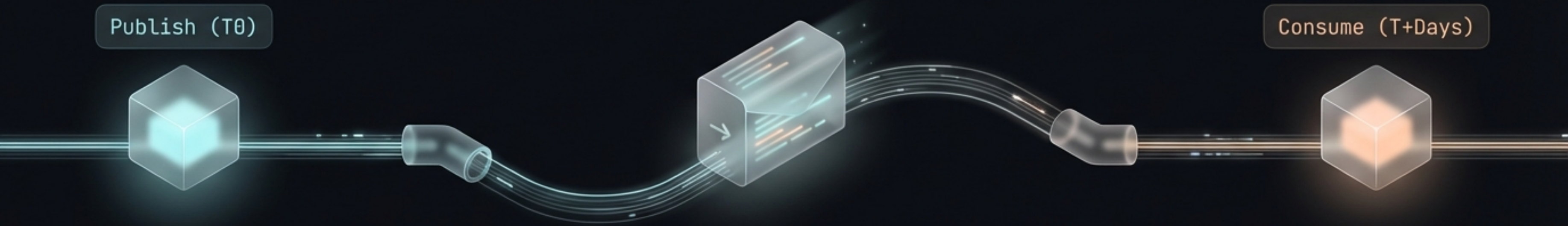


REST API та Публічний Контракт



- **Архітектура:** Domain Model $\neq$ DTO.
- **Принцип:** Приховування внутрішніх полів.
- **Обробка помилок:** Status Codes + JSON Details (частина контракту).
- **Стабільність:** API говорить стабільною мовою, поки домен еволюціонує.

Черги Повідомлень (Asynchronous Time Travel)



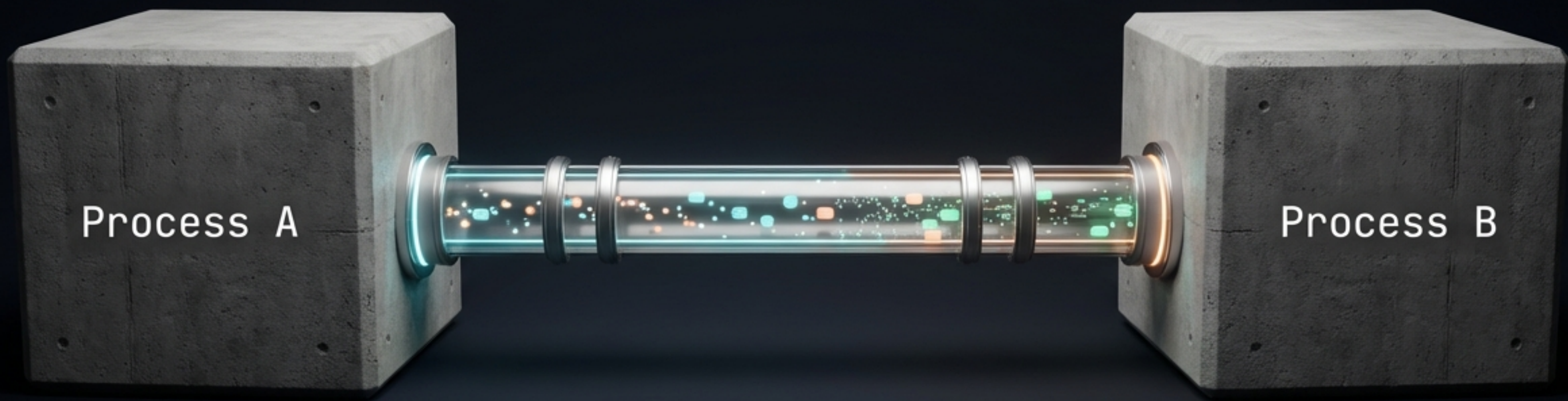
- Транспорт: `Kafka`, `RabbitMQ`, `NATS`.
- Фактор часу: Старі повідомлення мають бути читабельними (**Backward Compatibility**).
- Performance: **JSON** vs **Binary**. Масштабування трафіку.
- Governance: Інтеграція з `Schema Registry`.

Кешування (The Optimization Layer)



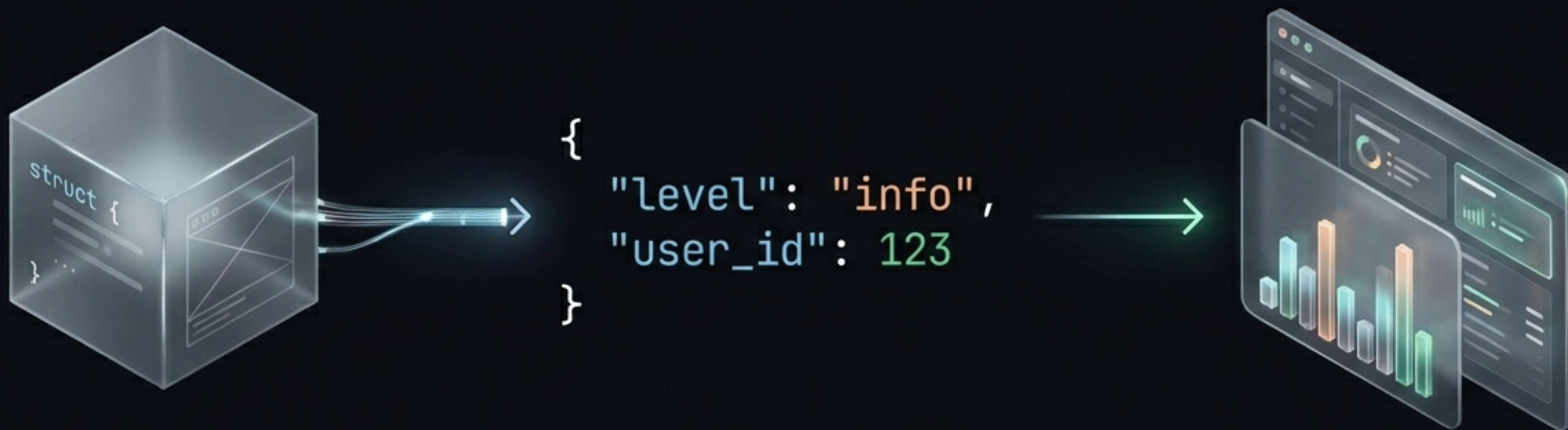
- Інструменти: `Redis`, `Memcached`.
- Ізоляція: Пам'ять процесів розділена – потрібна `серіалізація`.
- Ризик: `«Отруйні»` ключі після зміни структури (`panic`).
- Стратегія: `Versioned keys` або `інвалідація`.

Міжпроцесна Взаємодія (IPC & Isolation)



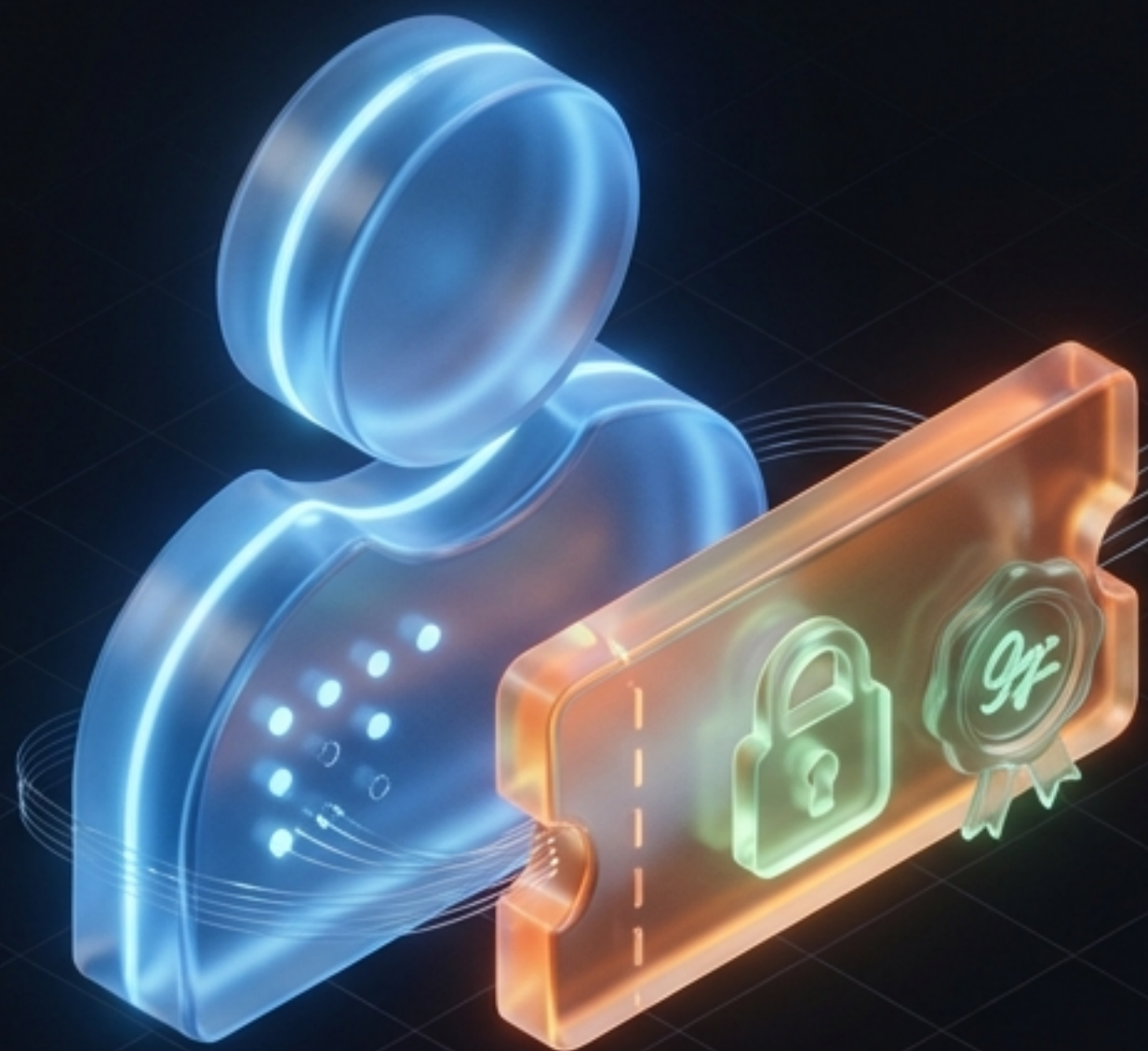
- Бар'єр: Адресні простори ізольовані. Вказівники не передаються.
- Механізми: **Pipe**, **UDS**, **Shared Memory**.
- **Shared Memory**: Вимагає контролю **layout**, **alignment**, **endianness**.
- Висновок: Серіалізація – міст між контекстами.

Логування (Observability Contract)



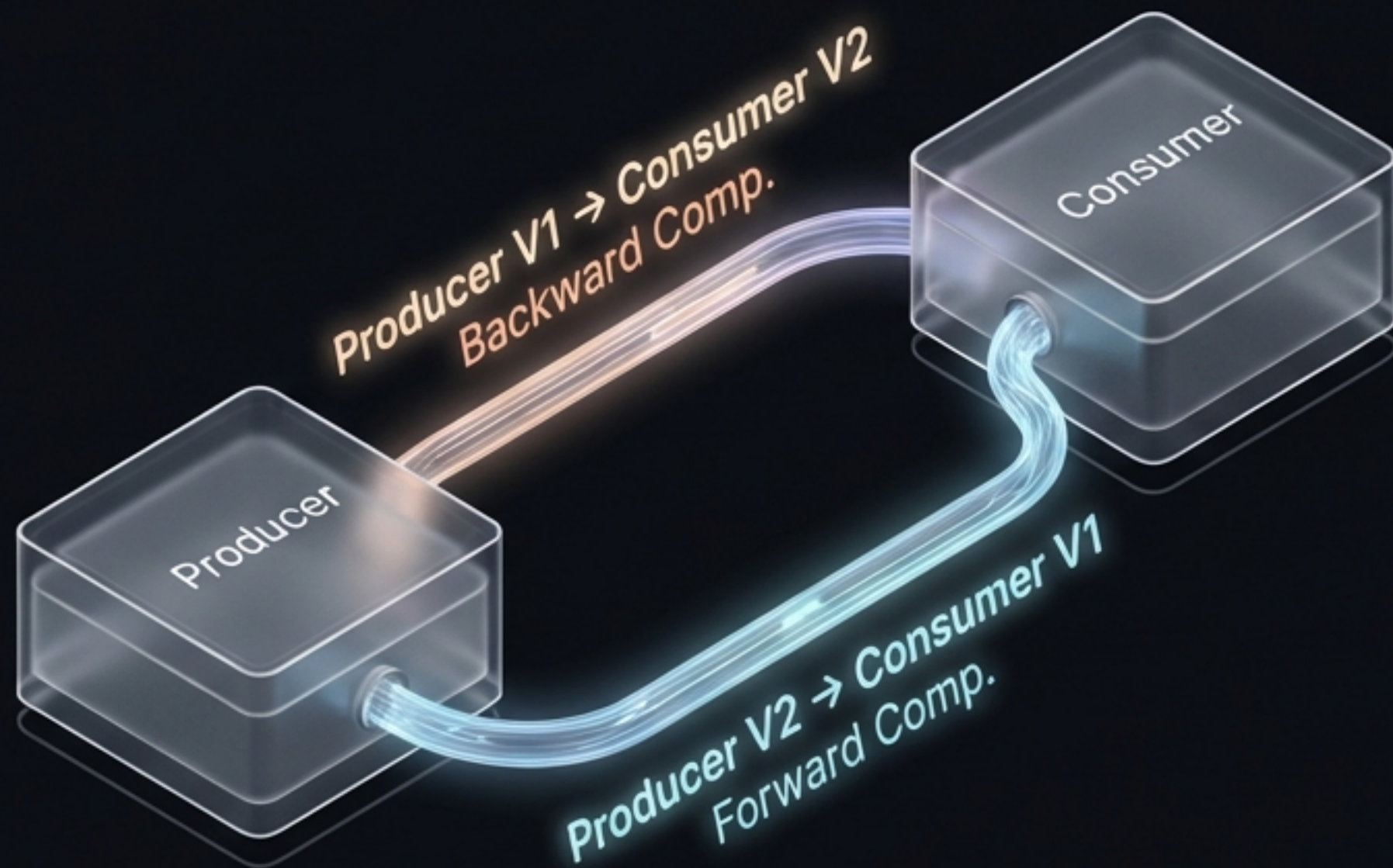
- Функція: Перетворення пам'яті в текст.
- Debug vs **Structured Logging** (JSON).
- **Observability**: Машиночитаність для фільтрації та аналітики.
- Privacy: Маскування PII.
- Контракт: Логи – це інфраструктурний API.

Сесії Користувача (State in Statelessness)



- Задача: HTTP — `stateless`.
Збереження стану між запитами.
- Зберігання: `Redis (bytes)` або `Cookie (signed JSON)`.
- Безпека: Десеріалізація — вектор атаки. Перевірка `підпису`.
- Еволюція: `Graceful fallback` при зміні структури.

Версіонування та Еволюція (Managing Change)



JetBrains Mono

- Реальність: Формати змінюються.
- Техніки: `Optional fields`, `default values`.
- Золоте правило: Ніколи не змінюйте тип поля (`int -> string`).
- Підхід: Інструмент керованої еволюції.

Реплікація (Distributed Consensus)



- Контекст: Distributed DBs, Consensus (Raft, Paxos).
- Вимога: Детермінізм. Node A == Node B.
- Rolling Updates: Робота зі змішаними версіями.
- Ефективність: Бінарні формати для зменшення лагу.